

Software Architecture and Design Specification

Project: File Compression Tool (Basic ZIP Implementation)

Version: 1.0

Authors: Team 05

Date: 03-10-2026

Status: Completed

Revision History

Version	Date	Author	Change Summary
1.0	03-10-2026	TEAM 5	Initial release of Software Architecture and Design Specification .

Approvals

Role	Name	Signature / Email	Date
QA Lead / Team Lead	Member 1	Karthik	Karthik - 03/10/2026
Developer	Member 2	Vaishnavi	Vaishnavi - 03/10/2026
Test Engineer	Member 3	Kavan	Kavan - 03/10/2026
Test Engineer	Member 4	Pavithra	Pavithra -03/10/2026

1. Introduction

1.1 Purpose

This document specifies the architecture and design of the File Compression Tool using Huffman Coding.

1.2 Scope

Covers file compression, Huffman code generation, and decompression.

1.3 Audience

Developers, testers, instructors, and maintenance teams.

1.4 Definitions

Huffman Coding, Compression, Decompression, Huffman Tree, Prefix-Free Code.

2. Document Overview

2.1 How to use this document

Describes the system architecture, UML diagrams, component design, and error handling.

2.2 Related Documents

SRS, STP, RTM.

3. Architecture

3.1 Goals & Constraints

Goals: Lossless compression, reliable decompression, and reduced file size.

Constraints: Memory limitations, file size, and processing time.

3.2 Stakeholders & Concerns

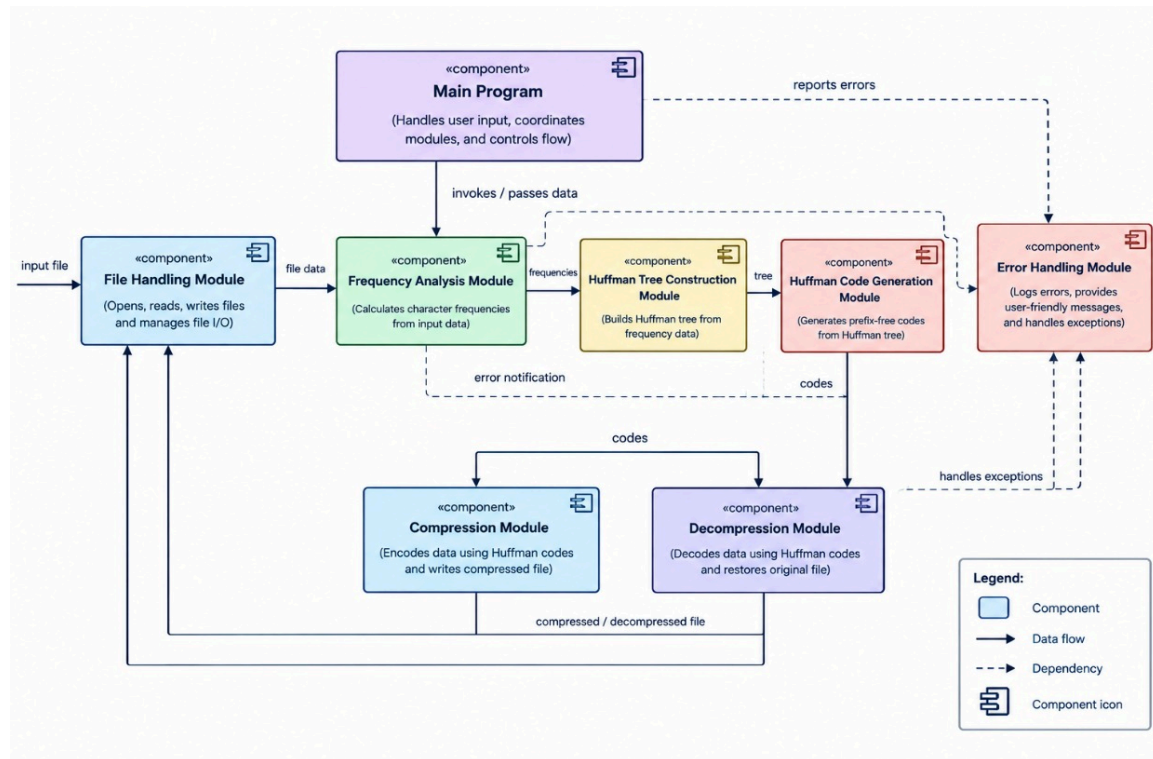
Users: File integrity and ease of use

Developers: Modularity and maintainability

Testers: Correct compression and decompression

Project Team: Reliability and performance

3.3 Component (UML) Diagram



3.4 Component Descriptions

Frequency Analysis Module: Calculates the frequency of each byte in the input file and generates a frequency table.

Huffman Tree Construction Module: Constructs a binary Huffman tree using the byte frequencies and a min-priority queue.

Huffman Code Generation Module: Traverses the Huffman tree and generates prefix-free binary codes for each byte.

Compression Module: Uses the generated Huffman codes to encode the input data into a compressed bitstream.

Decompression Module: Decodes the compressed bitstream using the Huffman tree or equivalent decoding information to reconstruct the original data.

File Handling Module: Reads input files and writes compressed and decompressed output files.

Main Program: Coordinates the modules and manages the command-line interface and overall execution.

3.5 Chosen Architecture Pattern and Rationale

Modular architecture is chosen to separate frequency analysis, Huffman coding, compression, decompression, and file handling. This makes the system easier to test and maintain.

3.6 Technology Stack & Data Stores

Programming Language: C/C++

Compression Algorithm: Huffman Coding

Input: Text or binary files

Output: Compressed and decompressed files

Data Storage: Local file system

3.7 Risks & Mitigations

Data Loss: Preserve the original input file during compression.

Corrupted Files: Validate compressed data during decompression.

Large Files: Handle file data efficiently using buffers.

Invalid Input: Display appropriate error messages.

3.8 Traceability to Requirements

FCT-F-003 (Frequency Calculation) → Frequency Analysis Module

FCT-F-004 (Huffman Tree Construction) → Huffman Tree Construction Module

FCT-F-005 (Huffman Code Generation) → Huffman Code Generation Module

3.9 Security Architecture

Threat Modeling (STRIDE): Spoofing → Input file validation; Tampering → File integrity checks; Information Disclosure → Avoid logging file contents; Denial of Service (DoS) → File size limits; Elevation of Privilege → Restrict file access permissions.

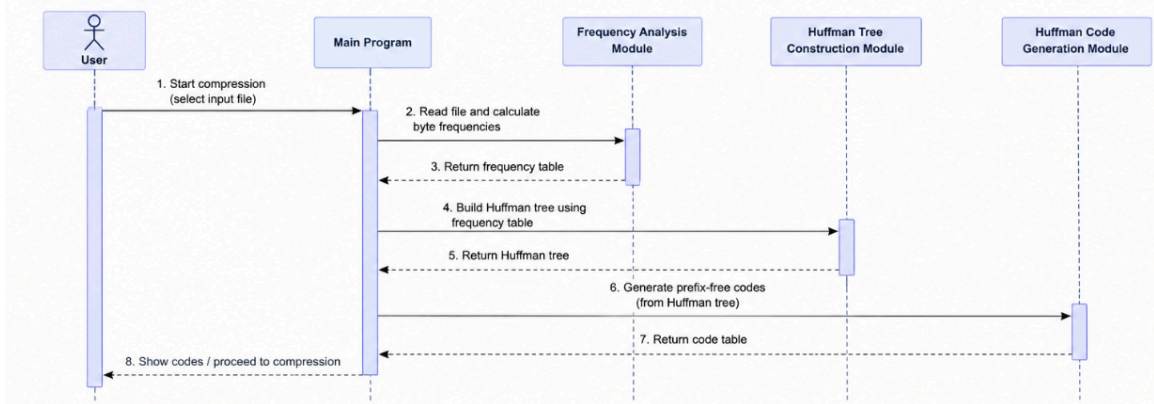
4. Design

4.1 Design Overview

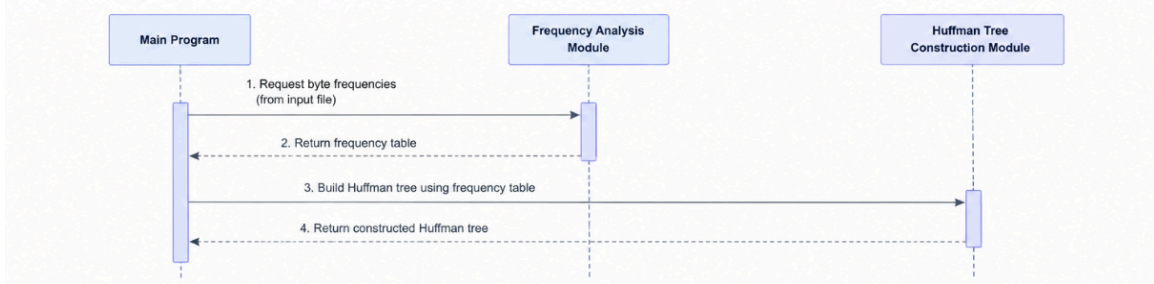
The File Compression Tool uses modular components for separation of concerns and easier maintenance.

4.2 UML Sequence Diagrams

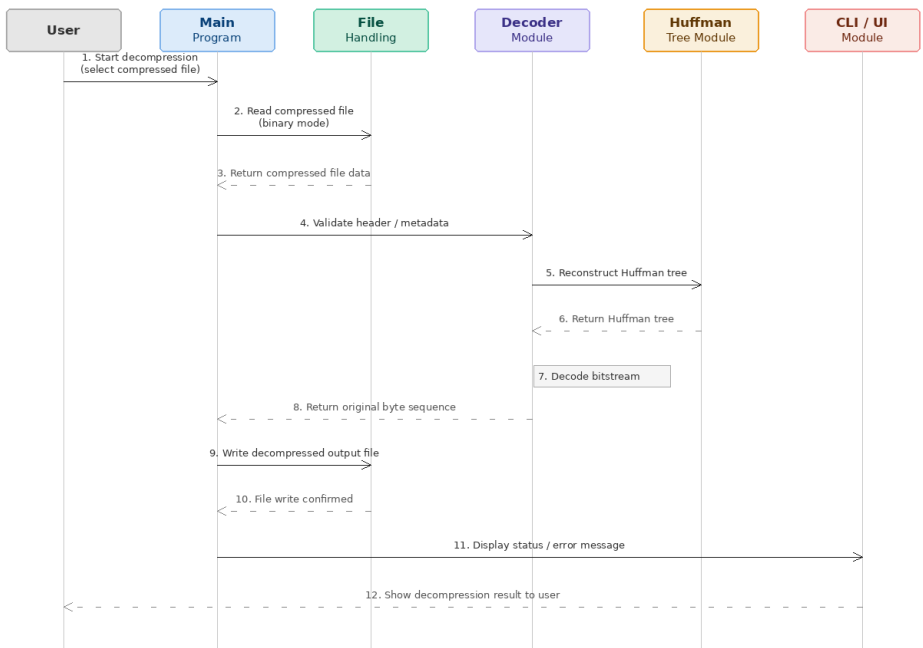
Sequence Diagram 1: Huffman Code Generation



Sequence Diagram 2: Huffman Tree Construction



Sequence Diagram 3: File Decompression Workflow



4.3 API Design

1. Frequency Analysis Module

- Function: `calculateFrequency`
- Input: Input file
- Output: Byte frequency table
- **Errors:** File not found, file read error.

2. Huffman Tree Construction Module

- Function: `buildHuffmanTree`
- Input: Byte frequency table
- Output: Constructed Huffman tree
- **Errors:** Empty frequency table, memory allocation failure.

3. Huffman Code Generation Module

- Function: `generateCodes`
- Input: Constructed Huffman tree
- Output: Code table
- **Errors:** Null/empty tree passed in, memory allocation failure during traversal.

4. Compression Module (Encoder)

- Function: `compressFile`
- Input: Input file path, output file path, code table
- Output: Compressed `.huff` file
- **Errors:** Input file not found or unreadable, output path invalid or unwritable, output file already exists and overwrite not confirmed, code table empty.

5. Decompression Module (Decoder)

- Function: `decompressFile`
- Input: Compressed `.huff` file path, output file path
- Output: Decompressed file written to output path
- **Errors:** Compressed file not found or unreadable, invalid/missing/corrupted header, incomplete or inconsistent metadata, output path invalid or unwritable, output file already exists and overwrite not confirmed, bitstream ends unexpectedly.

6. File Handling Module

- Function: `readFile`
- Input: File path (string), read mode (binary)
- Output: Raw byte buffer / stream of file contents
- **Errors:** File not found, insufficient read permissions, file is empty (returns empty buffer — caller decides whether to proceed).

- Function: `writeFile`
- Input: File path (string), byte buffer / data to write, write mode (binary)
- Output: File written successfully to disk
- **Errors:** Output path does not exist, insufficient write permissions, disk full, output file already exists (triggers overwrite check before writing).

7. Error Handling Module

- Function: `handleError`
- Input: Error code (enum), context message (string), optional: file path involved
- Output: Formatted error message printed to `stderr`; program exits with non-zero status code for critical errors, returns control to caller for recoverable ones
- **Errors:** N/A — this module is the error handler itself; it does not throw, it reports.

4.4 Error Handling, Logging & Monitoring

Standardized error messages for file access and compression failures. Logs record errors without sensitive file contents. Monitoring includes compression failure rate and successful file processing.

4.5 UX Design

User interface with clear file selection options, readable text, and simple controls for compression and decompression

4.6 Open Issues & Next Steps

Future enhancements: Support for additional compression algorithms, improved compression efficiency, and a graphical user interface.

5. Appendices

5.1 Glossary

- **Huffman Coding:** A lossless compression algorithm.
- **Huffman Tree:** A binary tree used to generate codes.
- **Prefix-Free Code:** A code in which no code is the prefix of another.

5.2 References

- Project Software Test Plan (STP).

- C/C++ documentation.

5.3 Tools Used

- C/C++
- Visual Studio Code
- UML diagram tool